PRÁCTICA GUIADA · SESIÓN 3

Mi primer script en Luz Clara

Enlazar un archivo .js al proyecto de clase, escribirlo línea por línea y leer los errores de la consola

Módulo	CSTI12004 Programación web en el entorno cliente
Unidad de aprendizaje	UA1 · Elaboración de componentes de software
Capacidad	1.1 Elaborar componentes de software mediante el lenguaje JavaScript, según los estándares de desarrollo vigentes.
Fecha	jueves 12 de noviembre de 2026 · presencial
Duración	50 minutos en clase, junto con la persona facilitadora
Herramientas	VS Code, Live Server, consola del navegador (F12) y Git
Persona facilitadora	Giovanni Antonio Coto Calderón

Cómo usar esta guía

- Se sigue **en clase**, al mismo ritmo de la persona facilitadora.
- Cada parte dice **qué vamos a hacer**, **cómo lo haremos** y el **resultado esperado**.
- Al final de cada parte, el código se explica **elemento por elemento**: cada palabra, símbolo y carácter nuevo.
- Guárdela: sirve de consulta para la práctica autónoma.

Antes de empezar · dos sitios, dos usos

En el M3 construyó dos sitios con HTML y CSS; en este módulo les agrega JavaScript. **En la práctica guiada todos trabajamos en Luz Clara**, el proyecto de clase: usted escribe exactamente lo que se ve en el proyector. **En la práctica autónoma aplica lo mismo en su Casa Tomillo**, con otros datos.

Carpeta	Sitio	Se usa en
`luz-clara/`	Luz Clara, el proyecto de clase del M3	Práctica guiada: igual que la persona facilitadora
`casa-tomillo/`	Casa Tomillo, su AC3 del M3	Práctica autónoma: usted aplica lo aprendido
`UNIDAD_1_javascript/CLASE_nn_fecha/`	Ejercicios cortos	Archivos con errores para corregir

Así debe quedar su repositorio al terminar hoy. Las flechas ← explican qué contiene cada carpeta o archivo:

Su repositorio al terminar la sesión 3

```
CSTI12004-Nombre-Apellido/
├── README.md
├── .gitignore
├── RECURSOS/                ← capturas de pantalla
├── luz-clara/               ← proyecto de clase del M3
│   ├── index.html
│   ├── galeria.html
│   ├── nosotros.html
│   ├── contacto.html        ← + ficha.js
│   ├── css/  recursos/
│   └── js/
│       └── ficha.js         ← esta práctica guiada
├── casa-tomillo/            ← su AC3 del M3
│   ├── index.html           ← + app.js y talleres.js
│   ├── cursos.html
│   ├── contacto.html
│   ├── css/  recursos/
│   └── js/
│       ├── app.js           ← prueba del entorno
│       └── talleres.js      ← práctica autónoma, ejercicio 1
└── UNIDAD_1_javascript/
    ├── CLASE_01_10-11-2026/  ← cuadro comparativo
    ├── CLASE_02_11-11-2026/  ← lo que hizo en la clase 2
    └── CLASE_03_12-11-2026/  ← horario/ y respuestas.md
```

Comprobación	Cómo saberlo
Los dos sitios están copiados	Dentro de luz-clara/ y de casa-tomillo/ están directamente los .html (no una carpeta extra con los archivos adentro).
Los sitios abren bien	Clic derecho en cada index.html → Open with Live Server: se ven con sus estilos e imágenes.
JavaScript está enlazado	En casa-tomillo/index.html, con F12 → Console, aparecen los dos mensajes de app.js.

Parte 1 · Ficha de un servicio → luz-clara/js/ficha.js

Qué vamos a hacer	Agregar JavaScript a una página de **Luz Clara**: un archivo **ficha.js** que muestra en la consola la ficha de la sesión de retrato individual, con los datos de la tabla de tarifas de contacto.html.
Cómo lo haremos	1) Enlazar el archivo al final de **contacto.html** con la etiqueta script. 2) Crear **luz-clara/js/ficha.js** y escribirlo **línea por línea**, igual que en el proyector, guardando y mirando la consola después de cada una. 3) Hacer commit y push.
Resultado esperado	Al abrir **contacto.html** con Live Server, la página se ve igual que siempre y la consola (F12) muestra la ficha del servicio y un aviso amarillo, sin errores rojos.

Cómo lo haremos · paso a paso

Comando (terminal de VS Code · Git Bash)	¿Qué hace?	Resultado
`cd /c/INA/CSTI12004-Nombre-Apellido`	Entra a la carpeta del repositorio.	(main)
`git pull`	Trae los cambios que subió en la S2.	Already up to date.

1. Abra **luz-clara/contacto.html** en VS Code y baje hasta el final del archivo.
2. Justo **antes de </body>**, escriba la etiqueta que enlaza el archivo:

luz-clara/contacto.html (final del archivo)

```
    <p class="pie-creditos"><small>&copy; 2026 · Estudio Luz Clara</small></p>
  </footer>

  <script src="js/ficha.js"></script>
</body>
```

Explicación de la etiqueta

```
<script src="js/ficha.js"></script>
```

Elemento	Qué es	Para qué sirve
`<script>`	Etiqueta de apertura de HTML.	Le avisa al navegador que aquí se carga un programa de JavaScript.
`src`	Atributo (de source, «origen»).	Dice dónde está el archivo .js que se debe cargar.
`=`	Signo igual.	Une el atributo con su valor.
`"…"`	Comillas dobles.	Encierran el valor del atributo.
`js/`	Nombre de carpeta seguido de una barra.	La carpeta js, que está junto a contacto.html. La barra / separa la carpeta del archivo.
`ficha.js`	Nombre del archivo.	Debe escribirse igual que el archivo real, con las mismas mayúsculas y minúsculas.
`.js`	Extensión del archivo.	Indica que el archivo contiene JavaScript.
`</script>`	Etiqueta de cierre.	Es obligatoria, aunque no haya nada entre las dos etiquetas. La / indica que cierra.
`antes de </body>`	Ubicación de la etiqueta.	El script se ejecuta cuando el navegador ya leyó todo el contenido de la página.

3. Guarde con **Ctrl+S**.
4. En el Explorador de VS Code, haga clic derecho sobre la carpeta **luz-clara/js** → **Nuevo archivo** (si la carpeta js no existe, créela: clic derecho sobre luz-clara → **Nueva carpeta** → js) → escriba **ficha.js**.
5. Escriba **las líneas 1, 2 y 3**, guarde y abra **contacto.html** con Live Server (clic derecho → **Open with Live Server**). Presione **F12** → **Console**: debe aparecer **=== ESTUDIO LUZ CLARA ===**. Si no aparece, revise la etiqueta del paso 2.

6. Escriba **una línea más** y, **antes de guardar**, diga qué cree que mostrará la consola. Guarde: Live Server recarga la página solo. Compare.
7. Repita el paso 6 hasta la línea 8.

luz-clara/js/ficha.js (completo)

```
1  // ficha.js - Ficha de un servicio (se enlaza al final de contacto.html)
2  /* Datos de la tabla de tarifas de contacto.html */
3  console.log("=== ESTUDIO LUZ CLARA ===");     // título
4  console.log("Sesión:\tIndividual");          // \t alinea el dato
5  console.log("Tipo:\t\"Retrato\"");           // \" muestra comillas
6  console.log("Precio:\t\u20A145 000");        // \u20A1 = ₡
7  console.log("Tiempo:\t1 hora\nLugar:\tSan José"); // \n salto de línea
8  console.warn("¡Quedan pocas fechas este mes!"); // aviso amarillo
```

Resultado esperado

Consola del navegador (ejecución real en Chromium)

```
=== ESTUDIO LUZ CLARA ===
Sesión: Individual
Tipo:   "Retrato"
Precio: ₡45 000
Tiempo: 1 hora
Lugar:  San José
⚠ ¡Quedan pocas fechas este mes!
```

La página **contacto.html** no cambia: la ficha solo se ve en la consola. Las líneas quedan alineadas gracias a **\t**.

Explicación del código, elemento por elemento

Cada elemento se explica la primera vez que aparece. En las líneas siguientes solo se explica lo nuevo.

Línea 1

```
// ficha.js - Ficha de un servicio (se enlaza al final de contacto.html)
```

Elemento	Qué es	Para qué sirve
`//`	Comentario de línea.	El navegador ignora todo lo que sigue hasta el final de la línea. Sirve para explicar el código a las personas.
`ficha.js - Ficha…`	Texto del comentario.	Por convención del curso, la primera línea dice qué archivo es y para qué sirve.

Línea 2

```
/* Datos de la tabla de tarifas de contacto.html */
```

Elemento	Qué es	Para qué sirve
`/*`	Apertura de un comentario de bloque.	Desde aquí el navegador ignora todo…
`*/`	Cierre del comentario de bloque.	…hasta aquí, aunque ocupe varias líneas.

Línea 3

```
console.log("=== ESTUDIO LUZ CLARA ===");      // título
```

Elemento	Qué es	Para qué sirve
`console`	Objeto del navegador que representa la consola (F12).	Indica dónde se va a escribir.
`.`	Punto.	Accede a algo que pertenece al objeto: «de console, usar log».
`log`	Método: una acción de console.	Escribe un mensaje en la consola.
`( )`	Paréntesis.	Encierran el dato que se le entrega al método (se llama argumento).
`" "`	Comillas dobles.	Marcan dónde empieza y dónde termina un texto. Un texto en programación se llama string.
`=== ESTUDIO LUZ CLARA ===`	Texto literal.	Se muestra tal cual. Dentro de las comillas, los signos = son solo caracteres.
`;`	Punto y coma.	Marca el fin de la instrucción.
`// título`	Comentario al final de la línea.	Explica qué hace la instrucción. No se muestra en la consola.

Línea 4

```
console.log("Sesión:\tIndividual");          // \t alinea el dato
```

Elemento	Qué es	Para qué sirve
`console.log( … );`	La misma instrucción de la línea 3.	Escribe un texto en la consola.
`\`	Barra invertida (dentro de un texto).	Avisa que el carácter siguiente tiene un significado especial. A esta pareja se le llama secuencia de escape. La barra no se muestra.
`\t`	Secuencia de escape: tabulación.	Salta a la siguiente columna. Sirve para alinear los datos uno debajo del otro.

Línea 5

```
console.log("Tipo:\t\"Retrato\"");          // \" muestra comillas
```

Elemento	Qué es	Para qué sirve
`\"`	Secuencia de escape: comilla doble.	Muestra una comilla dentro del texto. Sin la barra, esa comilla cerraría el texto antes de tiempo.
`"); al final`	Dos comillas seguidas.	La primera (con su barra) se muestra; la segunda cierra el texto.

Línea 6

```
console.log("Precio:\t\u20A145 000");         // \u20A1 = ₡
```

Elemento	Qué es	Para qué sirve
`\u`	Secuencia de escape Unicode.	Escribe un carácter usando su código.
`20A1`	Código del carácter (4 dígitos hexadecimales).	20A1 es el código del símbolo del colón: ₡.
`45 000`	Texto normal.	Se muestra justo después del ₡: ₡45 000.

Línea 7

```
console.log("Tiempo:\t1 hora\nLugar:\tSan José"); // \n salto de línea
```

Elemento	Qué es	Para qué sirve
`\n`	Secuencia de escape: salto de línea.	Lo que sigue se muestra en una línea nueva. Con un solo console.log se obtienen dos líneas.
`Lugar:\tSan José`	La parte del texto que sigue al \n.	Aparece en la línea nueva, también alineada con \t.

Línea 8

```
console.warn("¡Quedan pocas fechas este mes!"); // aviso amarillo
```

Elemento	Qué es	Para qué sirve
`warn`	Otro método de console (de warning, «advertencia»).	Muestra el mensaje como aviso: fondo amarillo y el ícono ⚠.
`¡ !`	Signos de exclamación.	Son parte del texto: se muestran tal cual.

Resumen de las secuencias de escape

- **\t** tabulación · **\n** salto de línea · **\"** comilla doble · \\ barra invertida · **\u20A1** símbolo ₡
- Siempre van **dentro de las comillas** de un texto. La barra invertida no se muestra.

Commit: git add luz-clara · git commit -m "S03: agrega ficha.js a contacto.html de Luz Clara" · git push

Parte 2 · Errores frecuentes

Qué vamos a hacer	Provocar **a propósito** los cuatro errores más comunes y aprender a leerlos en la consola, para reconocerlos cuando aparezcan sin querer.
Cómo lo haremos	Para cada error: 1) hacer **un solo cambio**, 2) guardar y mirar la consola, 3) leer en voz alta el **tipo**, el **mensaje** y el **archivo:línea**, 4) corregir y comprobar que todo vuelve a funcionar.

Resultado esperado	Cuatro mensajes distintos en la consola y, después de cada corrección, la consola otra vez sin errores rojos.

Parte del mensaje	Ejemplo	Qué indica
Uncaught	`Uncaught SyntaxError`	Un error que nadie atendió: el programa se detuvo.
Tipo	`SyntaxError · TypeError · ReferenceError`	La clase de error (ver la tabla del final).
Mensaje	`missing ) after argument list`	La descripción del problema, en inglés.
archivo:línea	`app.js:2`	Dónde está el problema. Es un enlace: al hacer clic se abre el archivo en esa línea.

Error 1 · La ruta del src

Cómo provocarlo: En **luz-clara/contacto.html**, cambie **ficha.js** por **fichas.js** (con una s de más). Guarde.

luz-clara/contacto.html · el src dice fichas.js

```
  </footer>

  <script src="js/fichas.js"></script>
</body>
```

Resultado esperado:

Consola del navegador (ejecución real en Chromium)

✕ `GET http://127.0.0.1:5500/js/fichas.js net::ERR_ABORTED 404 (Not Found)`

Parte del mensaje	Qué es	Qué significa aquí
`GET`	Pedido del navegador al servidor.	El navegador intentó descargar el archivo.
`…/js/fichas.js`	La dirección que pidió.	Es la ruta que escribió en el src.
`404 (Not Found)`	Código de respuesta: «no encontrado».	Ese archivo no existe en esa carpeta.

La página se ve bien, pero **el script no se cargó**. No es un error de JavaScript: el archivo ni siquiera llegó al navegador.

Corrección: Escribir el nombre exacto del archivo: **js/ficha.js**. Revise también la carpeta y las mayúsculas.

Error 2 · Una mayúscula

Cómo provocarlo: En una línea de su ficha.js, cambie **log** por **Log** (con L mayúscula). Guarde.

Ejemplo · errores/2_mayuscula/js/app.js

```
1  console.log("Primera línea: funciona");
2  console.Log("Segunda línea: Log con L mayúscula");
3  console.log("Tercera línea: ¿se ejecuta?");
```

Resultado esperado:

Consola del navegador (ejecución real en Chromium)

```
Primera línea: funciona
✕ Uncaught TypeError: console.Log is not a function   app.js:2
```

Parte del mensaje	Qué es	Qué significa aquí
`TypeError`	Error de tipo.	Se intentó usar como función algo que no lo es.
`console.Log is not a function`	«console.Log no es una función».	JavaScript distingue mayúsculas: log y Log son nombres distintos, y Log no existe.
`app.js:2`	Archivo y línea.	La línea 1 sí se ejecutó; en la 2 el programa se detuvo.

Las líneas **anteriores** se ejecutan; en la línea con el error el programa **se detiene** y las siguientes no se muestran.

Corrección: Escribir **console.log** en minúscula.

Error 3 · Comillas sin cerrar

Cómo provocarlo: En su ficha.js, borre la comilla de cierre de un texto (la que está antes del paréntesis). Guarde.

Ejemplo · errores/3_comillas/js/app.js

```
1  console.log("Primera línea: ¿se ejecuta?");
2  console.log("Segunda línea sin cerrar);
```

Resultado esperado:

Consola del navegador (ejecución real en Chromium)

```
✕ Uncaught SyntaxError: Invalid or unexpected token   app.js:2
```

Parte del mensaje	Qué es	Qué significa aquí
`SyntaxError`	Error de sintaxis.	Una regla de escritura está rota.
`Invalid or unexpected token`	«Elemento no válido o inesperado».	El navegador encontró el final de la línea dentro de un texto que nunca se cerró.
`app.js:2`	Archivo y línea.	Dónde empieza el texto sin cerrar.

No se ejecuta ninguna línea, ni siquiera las que están bien. Compárelo con el error 2, que sí dejó ejecutar la línea anterior.

Corrección: Cerrar la comilla: cada **"** de apertura necesita su **"** de cierre.

Error 4 · Comillas de Word

Cómo provocarlo: Copie un texto desde Word, un PDF o un chat y péguelo en el código: trae comillas curvas “ ”. Guarde.

Ejemplo · errores/4_tipograficas/js/app.js

```
1  console.log(“Hola desde Word”);
```

Resultado esperado:

Consola del navegador (ejecución real en Chromium)

✕ `Uncaught SyntaxError: Invalid or unexpected token   app.js:1`

Parte del mensaje	Qué es	Qué significa aquí
`SyntaxError`	Error de sintaxis.	Una regla de escritura está rota.
`Invalid or unexpected token`	«Elemento no válido o inesperado».	JavaScript no reconoce “ ni ” como comillas.
`“ ” frente a " "`	Comillas tipográficas frente a comillas rectas.	Solo las rectas, las del teclado en VS Code, marcan un texto.

A simple vista el código parece correcto. Fíjese en la forma de las comillas: las curvas son de procesadores de texto.

Corrección: Borrar las comillas curvas y escribirlas de nuevo **en VS Code**.

Para recordar

- **SyntaxError:** una regla de escritura está rota. **No se ejecuta nada** del archivo.
- **TypeError / ReferenceError:** el programa se ejecuta **hasta esa línea** y ahí se detiene.
- **404:** el archivo no se encontró. Revise el **src**.
- Siempre lea **tipo, mensaje y archivo:línea** antes de corregir.